

Limbajul pseudocod

D *Limbajul pseudocod* este o modalitate de reprezentare a algoritmului, independent de un anumit limbaj de programare. Acesta respectă reguli de scriere și o anumită sintaxă în descrierea operațiilor ce îl compun.

În descrierea algoritmilor prin limbaj pseudocod noțiunea de *operație* utilizată în cadrul schemei logice, ce va fi înlocuită cu noțiunea de *instrucțiune*. În realizarea algoritmului cu ajutorul limbajului pseudocod se utilizează aceleași structuri prezentate anterior: structura *secvențială*, structura *decizională* și structura *repetitivă*. Noțiunile utilizate până acum, date, variabile, constante, operatori, expresii, toate rămân valabile și pentru limbajul pseudocod.

Orice algoritm începe cu cuvântul *algoritm*, urmat de definirea acestuia în funcție de cerințe, și se termină cu notația *sfârșit algoritm*.



Exemplu:

algoritm sumă

.....

sfârșit algoritm

Structura *secvențială* conține următoarele instrucțiuni:

– *Declararea variabilelor sau a constantelor* – se realizează la începutul algoritmului și presupune specificarea pentru datele problemei a numelui, a tipului de date și a valorii primite pentru o constantă. Sintaxa instrucțiunii este:

Tipul <listă variabile>



Exemplu:

întreg a, b, c – s-au declarat trei variabile de tip întreg a, b și c

real m – s-a declarat o variabilă de tip real m

natural t – s-a declarat o variabilă de tip natural t

caracter x – s-a declarat o variabilă de tip caracter x

text $s \leftarrow „Azi e luni”$ – s-a declarat o variabilă de tip șir de caractere s , care are drept conținut textul dintre ghilimele

– *Instrucțiunea de citire* – preia succesiv valori și se asociază, în ordinea preluării, variabilelor specificate.

citește <listă variabile>

Exemplu:

citește a, b, c – s-au citit valorile pentru cele trei variabile **în ordinea înșirării lor**.

citește m – s-a preluat o valoare pentru variabila m .



Se pot prelua una sau mai multe valori folosind o singură instrucțiune de citire.

– *Instrucțiunea de scriere* – permite vizualizarea rezultatelor obținute în urma prelucrării datelor

scrie <listă variabile>



Exemplu:

scrie a, b, c – se afișează conținutul celor trei variabile a, b și c .

scrie „Media este”, m – se afișează mesajul urmat de valoarea variabilei m .



Se pot afișa atât conținuturile variabilelor, precum și anumite mesaje care se trec între ghilimele. Conținuturile și mesaje se separă prin *virgulă*.

– **Instrucțiunea de atribuire** – presupune atribuirea valorii obținute eventual în urma evaluării unei expresii, unei variabile din cadrul algoritmului

$v \leftarrow \text{expresie}$



Exemplu:

$b \leftarrow 13$ – i se atribuie variabilei b valoarea 13

$a \leftarrow b + 3$ – i se atribuie variabilei a rezultatul expresiei $b + 3$, și anume valoarea 16

$x \leftarrow (7 < a)$ – i se atribuie expresiei logice $(7 < a)$ rezultatul, și anume valoarea 1 (true)

$y \leftarrow (a < 3) \text{ AND } (b > 5)$ – i se atribuie expresiei logice $(a < 3) \text{ AND } (b > 5)$ rezultatul, și anume valoarea 0 (false)



În cadrul unei instrucțiuni de atribuire, tipul variabilei trebuie să corespundă cu tipul expresiei.

Structura **decizională (alternativă)** este descrisă în limbajul pseudocod cu ajutorul **instrucțiunii de decizie** și are sintaxa:

```

dacă (condiție) atunci
    instrucțiuni_1
altfel
    instrucțiuni_2
sf.dacă
  
```

Modul de execuție al instrucțiunii de decizie

- 1) Se evaluează **condiția**, stabilindu-se valoarea de adevăr a acesteia (Adevărat – True, Fals – False).
- 2) În funcție de valoarea de adevăr are loc una din cele două seturi de instrucțiuni. Dacă este **adevărată** condiția, se va executa setul de **instrucțiuni_1**, iar dacă aceasta este **falsă**, se va executa setul de **instrucțiuni_2**.
- 3) Indiferent de ramura pe care va merge, după executarea instrucțiunilor se iese din structură și se continuă algoritmul.

Există și o formă simplificată a acestei instrucțiuni, și anume aceea în care lipsește a doua ramură – **altfel**. În această situație avem următoarea formă:

```

dacă (condiție) atunci
    instrucțiuni
sf.dacă
  
```

Modul de execuție

- 1) Se evaluează **condiția**, stabilindu-se o valoare a acesteia (Adevărat – True, Fals – False).
- 2) Dacă este **adevărată** condiția, se va executa setul de **instrucțiuni**, iar dacă aceasta este **falsă**, se va ieși din structură.

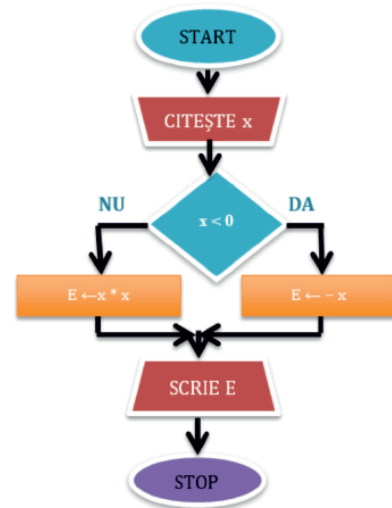


**Scrierea unui algoritm reprezentat prin schemă logică
în limbajul pseudocod. Parcurgerea unui algoritm**

Fie schema logică alăturată. Algoritmul scris în limbaj pseudocod este următorul:

```

algoritm expresie
    întreg  $x, E$       // se declară variabilele
    citește  $x$         // se citește  $x$ 
    dacă ( $x < 0$ ) atunci
         $E \leftarrow -x$     // ramura DA
    altfel
         $E \leftarrow x * x$  // ramura NU
    sf.dacă
    scrie  $E$            // se afișează valoarea expresiei
    sfârșit algoritm
  
```



Pentru a verifica ce returnează algoritmul, în cazul unei valori concrete citite, acesta trebuie parcurs liniar de la prima la ultima instrucțiune, iar în cazul instrucțiunii **dacă**, se evaluează condiția și se alege ramura pe care se continuă. După ce se trece de instrucțiunea **dacă**, se reia parcurgerea liniară a instrucțiunilor.

Pași parcurși:

- se declară variabilele x și E ;
- se citește o valoare pentru x ;
- dacă valoarea citită este -4 , atunci condiția ($x < 0$) este adevărată, deci E va lua valoare $-(-4)$, adică 4 ;
- dacă valoarea citită ar fi fost 5 , atunci condiția era negativă, iar E ar fi luat valoarea 25 ;
- se afișează valoarea lui E .